Coding with Claude Code

Yale Library | StatLab

2026-07-14

Workshop Flow

Segment	Time	Goal
Setup help	15 min	Get everyone running with a workshop key
Crash course	35 min	Mental model, security habits, prompts, context
Demo 1: Tidy a folder	15 min	Plan, approve, and act on real files
Demo 2: Fresh data	20 min	Download, check for updates, summarize
Demo 3: Verification	20 min	Skills and cross-language checking
Q&A	15 min	Connect the workflow to your own work

What You Should Be Able To Do

By the end, you should be able to:

1. Explain how Claude Code is different from a chat assistant.
2. Open Claude Code in the right working folder.
3. Write a prompt with a clear task, files, output, and done condition.
4. Review Claude's actions before trusting the result.
5. Know when to use Plan Mode, `/clear`, and project instructions.

The Big Idea

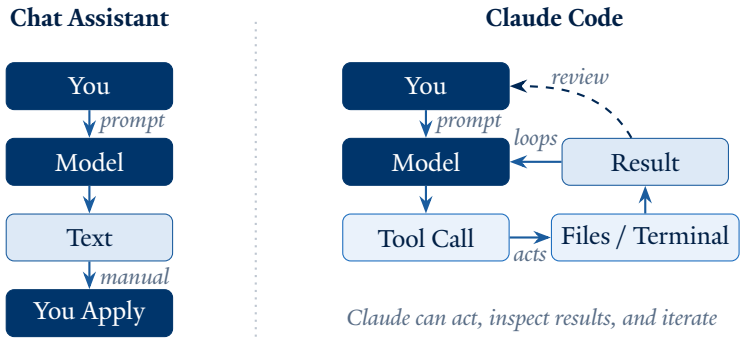
Claude Code is a coding agent.

That means it can:

- Read files in your project
- Edit or create files
- Run terminal commands
- Inspect output and revise its work
- Ask for permission before important actions

You are still responsible for the goal, judgment, and final review.

Chat Assistant vs. Coding Agent



Claude Desktop is useful for conversation. Claude Code is useful when the work lives in files.

Working Location Matters

Claude can only work with the location you give it.

Place	What Claude sees
Claude Desktop	Files you attach or upload
Terminal	The folder where you run `claude`
VS Code extension	The folder opened in VS Code
Positron extension	The folder opened in Positron

For this workshop, I will use the Claude Code extension inside Positron.

Security Hygiene: Four Rules

Claude Code is powerful because it can act. Four habits keep that power in check.

Rule	In one line
1. One project, one folder	Only open Claude Code in the folder it should touch
2. Keep your secrets	Never put API keys or PII where Claude can see them
3. Edit conservatively	Approve changes; avoid “YOLO mode” at all costs
4. Stop and ask	If you don’t understand what Claude is doing, interrupt and ask it

These four habits prevent nearly all of the ways an agent session goes wrong. Rules 1 and 2 next; rules 3 and 4 come later, once you have seen how Claude edits files and asks permission.

Security Rule 1: One Project, One Folder

The working folder sets the limits of what Claude can read and change.

- Never launch Claude Code from your home directory or `Documents`.
- Open the *specific* project folder before starting a session.
- One session, one project: switching projects means switching folders.
- If a task needs files from elsewhere, copy them in deliberately.

A small folder means small mistakes. A huge folder means Claude can touch everything in it.

Security Rule 2: Keep Your Secrets

Claude can read every file in the working folder, and everything you type is sent to the model provider.

Never let these appear in a prompt, in `CLAUDE.md`, or in project files:

- API keys, passwords, and access tokens
- Participant data or other PII (names, emails, IDs)
- Anything covered by an IRB protocol or data use agreement

Keep keys in environment variables or a `.env` file that lives outside the project (or is listed in `.gitignore`). De-identify data before it enters a Claude Code project.

A Good Prompt Has Four Parts

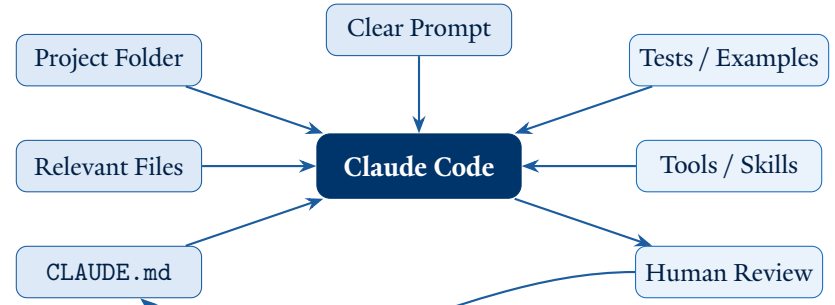
Part	Question to answer
Task	What do you want done?
Context	Which files, folders, or variables matter?
Output	What should Claude produce?
Done	How should Claude verify the result?

Write prompts the way you would brief a capable collaborator who is new to the project.

Weak Prompt, Strong Prompt

Weak	Strong
Analyze this data.	Read `data/input.csv`, summarize missingness by column, and save the result to `output/missingness.csv`.
Fix my script.	In `analysis/run_model.R`, find why the script fails, explain the issue, fix it, and rerun the script.
Make a plot.	Create a plot from `results/summary.csv`, save it to `figures/summary.png`, and report where the file was written.

Context Engineering is the New Prompt Engineering

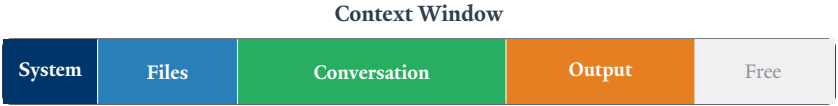


Better context reduces guessing: Claude can ground claims in files, tests, examples, and review.

Hallucinations are more likely when Claude has to infer missing context. Good context gives it evidence to inspect and checks to run.

Managing Your Context Window

Claude has a working memory for each session.

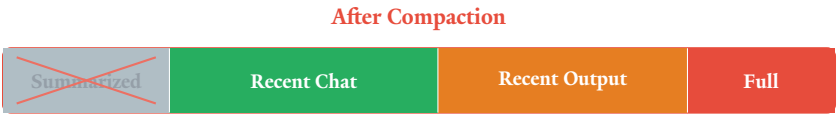


What fills context:

- Prompts and responses
- Files Claude reads
- Commands Claude runs
- Tool outputs and error messages
- Plans, edits, and verification results

Why Compaction Can Hurt

When context gets full, Claude compacts the session: detailed history is replaced by a shorter summary.



Earlier file details, intermediate decisions, and command outputs can be compressed or lost entirely.

The Compaction Trade-off

Performance Heuristic

$$\text{Performance} \approx \frac{\text{correctness}^2 \times \text{completeness}}{\text{size}}$$

Compaction reduces **size**, but it also risks reducing **correctness** and **completeness**. The numerator typically suffers more than the denominator gains.

The fix is not to avoid compaction, but to manage it intentionally:

- Compact **before** the window is full, not after
- Write key decisions and outputs to files **first** so they survive the summary
- Do not wait for auto-compaction; trigger it yourself with `/compact`

Context Commands and Habits

Command	Use it when…
`/context`	Check how much working memory remains
`/compact`	Mid-task and running low: compact and continue
`/clear`	Starting a new task: full clean slate

- Use one session for one main task
- Save outputs to files and write progress to `PLAN.md`; both survive `/clear`
- Put reusable project facts in `CLAUDE.md`

Tools

Tools are how Claude Code acts on your project.

Tool type	Example
Read files	Inspect scripts, data dictionaries, configs
Edit files	Modify code, write documentation, create outputs
Run commands	Execute R, Python, shell, tests, render steps
Search context	Find files, functions, errors, or patterns
Track tasks	Keep multi-step work organized

Claude chooses tools based on your prompt and the available context.

CLAUDE.md

`CLAUDE.md` is a project instruction file.

Place it at the root of your project:

```
my-project/
  CLAUDE.md
  data/
  scripts/
  output/
```

Claude Code reads it automatically when it starts in that project folder.

What Goes in CLAUDE.md

Good things to include:

- Project structure
- How to run the project
- Preferred packages or commands
- Output folders
- Naming conventions
- Common mistakes Claude should avoid

Keep it short. This is onboarding for the project, not a full manual.

CLAUDE.md Example

```
# Project Instructions

Use R scripts in `scripts/` for analysis work.
Save tables and figures to `output/`.

Before finishing a task:
- Run the script you changed.
- Report any warnings or errors.

Rules:
- Never overwrite files in `data/raw/`.
- Never read or write files containing credentials.
- Inspect data files with a script, never directly.
```

Start without one; add an instruction when you catch yourself repeating a correction.

Skills

A skill is a reusable procedure Claude can load when a task matches.

Skills are useful for:

- A repeated analysis or reporting workflow
- A required output format
- A review checklist
- A team convention
- A verification procedure

The idea is simple: write the procedure once, reuse it across projects.

Subagents

A subagent works in a separate context window.

Good uses:

- Review code that the main session just wrote
- Search a project for a specific pattern
- Investigate a narrow question in parallel
- Compare two possible approaches

For review tasks, give the subagent read-only instructions: flag issues, do not silently fix them.

Other Power-Ups

You do not need these on day one, but they are useful later.

Feature	Use it for
Plugins	Installing bundles of skills built by others
`/loop`	Repeating a task on a schedule
MCP servers	Connecting Claude to external tools
Hooks	Deterministic checks before or after actions

The core workflow comes first: prompt, inspect, act, verify.

Repeating Work: /loop

`/loop` runs a prompt or skill on a recurring interval.

```
/loop 30m if data/life-expectancy.csv is more than a
day old, refresh it and rerun the summary
```

Good uses:

- Keeping a dataset fresh during a work session (Demo 2)
- Rerunning `/tidy-folder` on a downloads directory (Demo 1)
- Watching a long-running job and summarizing when it finishes

Loops inherit your permission mode. Per Security Rule 3: a repeating task is the *last* place to skip permissions.

Plugins: Install Capabilities in One Step

A plugin bundles skills, subagents, hooks, and MCP connections that someone else already built.

1. Run `/plugin` and open the **Discover** tab.
2. Anthropic's official marketplace is preconfigured (catalog at claude.com/plugins).
3. Install a plugin; its commands appear namespaced, e.g. `/data:write-query`.
4. Its skills activate automatically when relevant.

Example: the official **data** plugin adds data profiling, SQL, statistical analysis, and validation checks. That is the plugin we used on the downloaded data in Demo 2.

Permission Modes

Use `Shift + Tab` to cycle modes.

Mode	What happens	Good for
Approval	Claude asks before edits and commands	New projects, risky changes
Auto-accept	File edits are accepted automatically	Fast iteration
Plan Mode	Claude reads and plans before changing files	Multi-step work

Start conservative. Speed up only when the task is low risk.

There is also a bypass flag that skips all permissions ("YOLO mode"). That is Security Rule 3, coming up: never use it.

Plan Mode

Plan Mode is the best place to course-correct.

Use it when:

- The task touches multiple files.
- You are not sure where the change belongs.
- You want Claude to explain the approach before acting.
- The cost of a wrong edit would be annoying to unwind.

Cycle into Plan Mode with `Shift + Tab`.

Security Rule 3: Edit Conservatively (No YOLO Mode)

Now that you have seen the permission modes, here is Rule 3 in full. Claude Code has a bypass flag (`--dangerously-skip-permissions`, "YOLO mode") that removes every confirmation prompt.

Do not use it. The permission prompts are your checkpoint between what Claude proposed and what Claude actually did.

Instead	When
Approval mode (default)	New projects, anything touching data
Plan Mode	Multi-step or unfamiliar work
Auto-accept edits	Only low-risk, easily reversible iteration

Security Rule 4: Stop and Ask

If you do not understand what Claude is doing: **STOP AND ASK IT.**

- Press `Esc` to interrupt Claude mid-action at any time.
- Ask: *"Explain what you were about to do and why."*
- Only approve an action once the explanation makes sense to you.

You are the one approving actions, so you are responsible for understanding them. Claude is very good at explaining itself. Make it do so.

Choosing a Claude Model

Claude Code supports model aliases, so you do not need to remember exact model IDs.

Alias	Current model	Use it for
`haiku`	Haiku 4.5	Fast chores: renames, small edits (Demo 1)
`sonnet`	Sonnet 5	Everyday analysis, debugging (Demos 2–3)
`opus`	Opus 4.8	Hard planning, deep review, verification

Use `/model` during a session or start Claude with `claude --model sonnet`.

Workshop API key users: models go by OpenRouter name (we will set this together). Watch usage with `/cost`.

Do Not Use It as a Flotation Device

Claude Code is helpful, but it is not a substitute for understanding the work.

Good fits:

- Tedious file edits and project navigation
- Drafting scripts from clear requirements
- Debugging with visible error messages
- Creating repeatable checks or reports

Weaker fits:

- Vague goals with no success criterion
- Work you cannot evaluate yourself
- High-stakes changes without tests or review
- One-shot “make this correct” requests

Build Trust with Benchmarks

Whether agentic AI is an appropriate tool is an *empirical question*:

1. Start with a small task where you know the expected result.
2. Run the same task manually or with an existing script.
3. Compare human and AI outputs.
4. Turn the comparison into a test, check, or verification skill.
5. Expand the workflow only after it passes benchmarks.

The goal is an improving toolchain, not a single impressive answer.

Review Before You Trust

Pause at natural checkpoints:

- Did Claude use the right folder?
- Did it read the right files?
- Did it understand the task?
- Did it change only what you expected?
- Did it run the relevant checks?
- Can you explain the result yourself?

The useful workflow is not autopilot. It is delegation with review.

A Reliable Workflow

Use the same rhythm for most Claude Code work:

Step	What you do
Explore	Ask Claude to inspect the project and summarize what it sees
Plan	Use Plan Mode for multi-step work
Act	Let Claude make the smallest useful change
Verify	Run the relevant command, test, render, or check
Save	Keep useful outputs in files, not only in chat

Demo Roadmap

Three demos, from everyday chores to real analysis.

Demo	Task	What it shows
1. Tidy a folder	Organize a messy project folder	Plan, approve, act on files
2. Fresh data	Download and summarize Our World in Data	Fetch, check for updates, analyze
3. Verify	Replicate a FRED analysis in a second language	Skills and verification habits

Same rhythm every time: prompt, inspect, act, verify.

Demo 1: Tidy a Project Folder

The first demo is a daily research task, not a coding task.

We start with a folder every researcher recognizes: `final_FINAL_v3.R`, `data (1).csv`, stray notes, no structure.

We will ask Claude to:

1. Inventory every file in the folder.
2. Propose a folder structure and naming scheme (Plan Mode).
3. Wait for approval.
4. Move and rename the files.
5. Write a `README.md` describing the new layout.

No code and no data analysis. Just delegation with review.

Demo 1 Prompt Pattern

```
Look at every file in this folder.

Propose a folder structure (data, scripts, notes,
archive) and a consistent naming scheme.

Wait for my approval, then move and rename the
files and write a README.md describing the layout.

Do not delete anything.
```

Note the three protections in the prompt: propose first, wait for approval, never delete.

Demo 1: From Chore to Automation

Once a task like this works, you never have to describe it again.

- Save the procedure as a skill (e.g. `/tidy-folder`) and reuse it on any folder.
- Recurring upkeep can run on a schedule with `/loop` (more later).
- Other chores that fit this pattern: renaming survey exports, archiving old drafts, standardizing figure names, splitting a notes file.

If you find yourself doing a file chore weekly, that chore is a candidate for automation.

Follow-Along Checkpoints

During the demo, pause and check:

1. What folder is Claude working in?
2. What files did Claude inspect?
3. What command did Claude run?
4. What file did Claude create or modify?
5. What evidence shows the task is done?

These same questions work for almost any Claude Code task.

Demo 2: Download, Check, Summarize

The second demo works with real, live data from Our World in Data.

We will ask Claude to:

1. Check whether `data/life-expectancy.csv` exists and how old it is.
2. Download or refresh it from the OWID URL if it is missing or stale.
3. Summarize the data: highest, lowest, and trend over time.
4. Save a summary table and a figure to `output/`.

The data is just country, year, and value: simple enough to check by eye, which is what a first data task should be.

Demo 2 Prompt Pattern

```
If data/life-expectancy.csv is missing or more than
30 days old, download a fresh copy from
https://ourworldindata.org/grapher/life-expectancy.csv

Then summarize it: the 5 highest and 5 lowest
countries in the most recent year, and the global
trend since 1950.

Save a table to output/tables/ and a plot to
output/figures/, and report what you did.
```

Demo 2: Keep the Data Out of the Chat

Reading a whole data file into the conversation is slow, costs tokens, fills the context window, and can expose sensitive rows.

The demo project protects against this in two layers:

Layer	How it works
`CLAUDE.md` rule	"Inspect data files with a script, never by reading them directly." Claude follows the instruction.
Hook	A `PreToolUse` hook blocks any direct read of `.csv` and similar files, even if Claude forgets.

Instructions are advice; hooks are enforcement. Use both for anything that matters. Claude instead writes a short script that prints the dimensions, columns, and summary, and reads that output.

Demo 2: An Optional Power-Up (the Data Plugin)

Plugins add ready-made skills in one step (details later).

Live extension of this demo:

1. Run `/plugin`, open the Discover tab, and install Anthropic's **data** plugin.
2. Ask Claude to *profile and validate* the downloaded CSV.
3. Watch the plugin's data-quality skills activate automatically.

The core workflow needs no plugins. A plugin packages expertise, like these profiling and validation checks, so you do not have to write it yourself.

Demo 3: Verification Pattern

The third demo shows a verification habit, using a simple FRED series (US unemployment rate: one date column, one value column).

We will ask Claude to:

1. Download the series from FRED and run an analysis in R.
2. Identify the key numerical outputs.
3. Reproduce the result another way.
4. Compare the outputs.
5. Report whether they match.

This is where skills or subagents become useful: they make review repeatable.

Demo 3: Cross-Language Verification

One useful verification pattern is to reproduce an analysis in a second language.

The idea:

1. Run the original analysis.
2. Identify the key numerical outputs.
3. Recreate the analysis in another language.
4. Compare the outputs.
5. Flag any mismatch for human review.

A match is not a proof, but it is a strong check against transcription errors, package defaults, and silent mistakes.

Demo 3: Skill File

A skill is a `SKILL.md` file: YAML frontmatter on top, instructions below.

```
---
name: cross-lang-verify
description: Verify analysis results by replicating in a
  second language and comparing numerical outputs.
model: opus
context: fork
---
You are a rigorous statistical programmer conducting a
formal verification audit. Replicate faithfully and flag
discrepancies rather than explain them away.
```

Frontmatter controls *when and how* the skill runs: `description` tells Claude when it is relevant, `model` picks who runs it, `context: fork` runs it in a fresh subagent. The body carries the *persona and procedure*.

Demo 3: Skill Procedure

The body of the skill gives Claude the steps to follow.

```
## Procedure
1. Identify the primary outputs to verify
2. Choose the target language
   - R analysis -> Python replication
   - Python analysis -> R replication
3. Write the replication to verify_<script>.<ext>
4. Run both versions
5. Compare key outputs to 4 decimal places
6. Report VERIFIED or MISMATCH with exact values
```

This turns a review habit into a reusable workflow.

Demo 3: Where the Skill Lives

A skill is just a folder with a `SKILL.md` inside your project.

```
project-folder/
  .claude/
    skills/
      cross-lang-verify/
        SKILL.md
```

In Positron, you can show this in the file explorer:

1. Open the project folder.
2. Create a `.claude` folder if it does not exist.
3. Inside it, create `skills/cross-lang-verify/`.
4. Save the skill as `SKILL.md` in that folder.

The dot in `.claude` means "project configuration folder." The folder name (`cross-lang-verify`) becomes the command: `/cross-lang-verify`.

Demo 3: Let Claude Create It

If you are terminal-shy, ask Claude Code to create the file for you.

```
Create a skill at
.claude/skills/cross-lang-verify/SKILL.md
for cross-language verification.

Use the procedure from the open SKILL.md example.
```

Then review the file in the Positron file explorer before using it.

The important habit: Claude can create project files, but you still inspect what it wrote.

Demo 3: Using the Skill

Once the file exists, invoke it directly:

```
Use the cross-lang-verify skill to verify this analysis.
```

Or ask naturally:

```
Verify this analysis by reproducing the key outputs in a second language.
```

Claude uses the skill because the request matches the skill description.

Demo 3: What We Will Ask

In the live demo, Claude will:

1. Download the FRED unemployment series and analyze it in R (mean, recent trend, plot).
2. Record the key outputs.
3. Write a Python replication using the skill.
4. Compare the results to 4 decimal places.
5. Report VERIFIED or MISMATCH.

Q&A

Useful questions to bring back to your own work:

1. What is a low-risk task I could try first?
2. What files would Claude need to see?
3. What output would count as done?
4. What checks would I require before trusting it?

Claude Code is most useful when you combine clear delegation with careful review.

Appendix: Quick Reference

Action	How
Start Claude Code	`claude`
Change model	`/model` or `claude --model sonnet`
Cycle modes	`Shift + Tab`
Check context	`/context`
Compact and continue	`/compact`
Start fresh	`/clear`
Check connection and login	`/status`
Browse and install plugins	`/plugin`
Get help	`/help`
Exit	`/exit`

Use this as a memory aid, not as a list to memorize.

Appendix: Customization Reference

Feature	Where it lives	Use it for
`CLAUDE.md`	Project root	Project instructions Claude reads automatically
Skills	`.claude/skills/<name>/`	Reusable procedures
Subagents	Managed with `/agents`	Focused review or parallel investigation
Plugins	Managed with `/plugin`	Installing bundles of skills and connections
Recurring tasks	`/loop <interval> <task>`	Repeating a prompt or skill on a schedule
MCP servers	Managed with `/mcp`	Connections to external tools
Hooks	Managed with `/hooks`	Deterministic commands at lifecycle events

Appendix: Claude Code and Codex

Both are coding agents. Use the one that is easiest to verify for the task.

Task	Claude Code	Codex
Live teaching or workshop demo	Strong fit in terminal/editor	Strong fit if already using Codex workflow
Local project exploration	Strong fit in an opened folder	Strong fit with CLI or IDE workflows
Reusable project instructions	`CLAUDE.md`, skills, subagents	Project instructions and review workflows
Independent verification	Useful as one checker	Useful as another checker

Best practice: compare outputs across tools, tests, or human review instead of trusting any single agent.